

Real-Channel GMSK SNR vs BER Simulation

This folder contains a MATLAB simulation chain for measuring the bit error rate (BER) of a GMSK burst waveform over a GSM-style fading channel plus AWGN. The code is organized as a complete transmit-channel-receive loop:

1. Read payload bytes from an AXI4-Stream-like CSV file.
2. Build a burst containing preamble, convolutionally encoded payload, and training bits.
3. Modulate the burst using the local Gaussian filter and GMSK mapper.
4. Pass the complex baseband signal through a GSM hilly-terrain fading channel and add AWGN at a selected SNR.
5. Detect bursts, equalize the complex samples using known preamble/training samples, demodulate to soft metrics, Viterbi-decode the payload, and write recovered bytes back to CSV.
6. Compare transmitted and received bytes or repeat this process over an SNR sweep to estimate BER.

The main scripts are `main_system.m` for one end-to-end run and `plot_ber_vs_snr.m` for a BER sweep.

Requirements

The code uses MATLAB System objects and communications/DSP functionality, including:

- `matlab.System`
- `dsp.FIRFilter`
- `comm.RaisedCosineTransmitFilter` in the standalone MathWorks LMS example
- `stdchan`
- `physconst`
- `readtable` / `writetable`

The project assumes the current working directory is this `real_curve` folder when running scripts, because several scripts use relative filenames such as `tx_data.csv` and `rx_data.csv`.

High-Level Data Flow

```
tx_data.csv
|
v
read_axi_stream_csv.m
|
v
tx_chain.m
|-- Waveform.m
|-- convEncoder.m
|-- build_packet.m
|-- GaussianFilter.m
|-- GmskMapper.m
v
complex TX samples
|
```

```

    v
channel_model.m
    |
    v
complex RX samples
    |
    v
rx_chain.m
    |-- RxDemodulator.m
    |-- GmskDemapper.m
    |-- detect_preamble.m
    |-- gmsk_modulate_burst_ref.m
    |-- preamble_sample_indices.m
    |-- lms_equalizer_complex.m
    |-- demod_aligned_burst.m
    |-- viterbi_soft_decoder.m
    |-- write_axi_stream_csv.m
    v
rx_data.csv
    |
    v
compare_payload_bytes.m

```

For BER curves, `plot_ber_vs_snr.m` wraps the same transmit, channel, receive, and compare process in a Monte Carlo loop over `SNR_dB_range`.

Main Entry Points

`main_system.m`

Single-run integration script. It:

- Sets `inputFile = 'tx_data.csv'`.
- Calls `tx_chain` to build and modulate one transmit burst.
- Calls `channel_model` at a fixed `SNR_dB = 24`.
- Calls `rx_chain` to detect, equalize, demodulate, decode, and write `rx_data.csv`.
- Calls `compare_payload_bytes` to compare `tx_data.csv` and `rx_data.csv`.

Use this when checking whether the full link works for one SNR value.

`plot_ber_vs_snr.m`

BER sweep script. It:

- Sets `SNR_dB_range = 0:2:30`.
- Sets `MIN_ERRORS = 1000`, `MAX_BITS = 1e5`, and `targetBER = 1e-5`.
- Builds one transmit waveform using `tx_chain`.
- Repeatedly calls `channel_model` and `rx_chain` at each SNR.
- Converts decoded bytes back to bits and compares them with the original payload bits.
- Plots BER vs SNR on a semilog y-axis.
- Estimates the SNR where the measured curve crosses `targetBER`.

One implementation detail matters: `totalBitErrors` is initialized to `1` for each SNR. This avoids zero-BER points on the semilog plot but slightly biases the BER upward, especially at high SNR when few or no real errors occur.

`sweep_equalizer_params.m`

Parameter sweep utility for the complex LMS equalizer. It tests combinations of:

- LMS step size `mu`
- equalizer length `eq_len`
- equalizer decision delay `decisionDelay`

For each combination, it runs the receiver on one detected burst and reports byte errors, bit errors, and final LMS MSE. Results are sorted so the best parameter combinations appear first. This script is useful for tuning `rx_chain.m`, whose current equalizer defaults are:

- `mu = 0.05`
- `eq_len = 13`
- `decisionDelay = floor((eq_len - 1) / 2)`

Transmitter Files

`tx_chain.m`

Top-level transmitter function:

```
[txSignal, waveform, signalPower] = tx_chain(csvInputFile)
```

Responsibilities:

- Reads input bytes using `read_axi_stream_csv`.
- Creates the waveform configuration using `Waveform('waveformId', 0)`.
- Pads or truncates the payload to `waveform.pldLenInfBytes`.
- Creates the convolutional encoder, Gaussian filter, and GMSK mapper.
- Calls `build_packet` to create the full burst bit sequence.
- Filters and GMSK-modulates the burst.
- Computes average signal power for later channel noise scaling.

`Waveform.m`

Defines the burst format and waveform constants. For `waveformId = 0`, it sets:

- Preamble length: `48` bits
- Training length: `32` bits
- Information payload length: `34` bytes
- Encoded payload length: `544` bits
- Code rate: `1/2`
- Burst length: preamble + encoded payload + training

It also builds `bitMapMask`, where:

- `1` means preamble
- `2` means payload
- `3` means training sequence

The payload and training sequence are interleaved in four chunks:

```
Preamble | Payload1 | Training1 | Payload2 | Training2 |  
Payload3 | Training3 | Payload4 | Training4
```

Important note: `makeRandomSeq` calls `rng(12345)` every time it creates a sequence. This makes the preamble and training deterministic and repeatable. Because the seed is reset for each sequence, the generated preamble and training patterns may be correlated or identical over overlapping lengths depending on sequence length. The receiver depends on this deterministic behavior.

`build_packet.m`

Creates the transmit burst bit vector. It:

- Converts payload bytes to MSB-first bits.
- Encodes them using `convEncoder`.
- Inserts the preamble.
- Splits the encoded payload into four chunks.
- Inserts four training chunks between payload chunks.

The output length is `waveform.burstBlockLen`.

`convEncoder.m`

Rate-1/2 convolutional encoder implemented as a MATLAB System object. It uses:

- Constraint length `K = 7`
- Generator polynomials `[171 133]` in octal

For each input bit, it produces two encoded output bits.

`GaussianFilter.m`

Gaussian pulse-shaping filter implemented as a MATLAB System object. It:

- Repeats each bit by `SPS`.
- Maps bits from `{0,1}` to NRZ symbols `{-1,+1}`.
- Convolves with a Gaussian impulse response.
- Quantizes coefficients to Q15.
- Saturates the filtered output to signed 16-bit values.

Default transmitter settings are:

- `BT = 0.35`

- `SPS = 4`
- `spanSymbols = 8`

GmskMapper.m

Maps Gaussian-filtered values to complex GMSK samples. It:

- Reduces filtered-bit gain using `bitGainReduct`.
- Accumulates phase in an integer accumulator.
- Wraps phase into a 512-entry sine/cosine lookup table.
- Outputs complex samples as `double(I) + 1i*double(Q)` using Q15 LUT values.

The mapper intentionally models a fixed-point/LUT-style implementation rather than an ideal floating-point continuous phase modulator.

Channel File

channel_model.m

Applies the real-channel impairment model:

- Samples per symbol: `SPS = 4`
- Symbol rate: `270.833 ksym/s`
- Carrier frequency: `1.8 GHz`
- Velocity: `0 m/s`
- Channel profile: `stdchan('gsmHTx12c1', Rsamp, fd)`
- AWGN scaled according to the requested `SNR_dB`

The faded signal is normalized back to the original transmit signal power before noise is added.

Receiver Files

rx_chain.m

Top-level receiver function:

```
allDecodedBytes = rx_chain(rx_full)
```

Responsibilities:

- Creates the same `Waveform` configuration used by the transmitter.
- Creates TX-like Gaussian/GMSK objects for reference-signal generation.
- Creates the receiver demodulator.
- Demodulates the full received sample stream to coarse symbol metrics.
- Detects the preamble using `detect_preamble`.
- Reconstructs an ideal complex reference burst for the known preamble/training bits using `gmsk_modulate_burst_ref`.
- Extracts each detected burst from the complex receive stream.
- Normalizes burst amplitude.

- Runs `lms_equalizer_complex` on complex samples before GMSK demapping.
- Demodulates the equalized burst using `demod_aligned_burst`.
- Extracts payload soft metrics using `waveform.bitMapMask == 2`.
- Decodes the payload with `viterbi_soft_decoder`.
- Packs decoded bits into bytes.
- Writes decoded bytes to `rx_data.csv` using `write_axi_stream_csv`.

The receiver uses equalization before demapping. This matters because the GSM fading channel distorts the complex baseband waveform, not just the final bit metrics.

RxDemodulator.m

Receiver demodulator System object. It:

- Calls `GmskDemapper` to compute phase differences.
- Samples the demapped stream at symbol times.
- Applies group-delay-aware symbol indexing based on Gaussian filter span and samples per symbol.

This is used for coarse full-stream preamble detection. After burst extraction and equalization, `rx_chain.m` uses `demod_aligned_burst.m` instead to avoid applying group-delay trimming a second time.

GmskDemapper.m

Converts complex GMSK samples into instantaneous phase-change metrics. It:

- Computes phase with `atan2`, or `cordic_atan_mex` if available.
- Unwraps the phase.
- Differentiates phase to produce `dphi`.

The output is a soft metric stream. Positive and negative values are later used by preamble detection and the soft Viterbi decoder.

detect_preamble.m

Detects packet starts by correlating soft demodulator metrics against the known preamble. It:

- Converts the preamble from bits to bipolar values.
- Computes normalized correlation using convolution.
- Finds peaks above a threshold.
- Suppresses nearby duplicate detections using `minSpacing`.
- Returns detections sorted by stream index.

Current `rx_chain.m` call:

```
detect_preamble(llr_full, double(waveform.prmSeq), 0.7, 500, true)
```

gmsk_modulate_burst_ref.m

Builds the ideal complex reference signal used by the LMS equalizer. This is a critical receiver helper.

It takes a burst where known positions are filled with the preamble and training bits, while unknown payload positions are set to zero. It then runs that burst through the same `GaussianFilter` and `GmskMapper` objects used by the transmitter, trims Gaussian filter group delay, and normalizes the result to unit amplitude.

It returns:

- `s_ref`: ideal unit-amplitude complex reference samples
- `trainSampleIdx`: sample indices corresponding to training bits
- `burstLenSamples`: nominal burst length in samples

The purpose is to give the equalizer a reference that matches the actual local fixed-point GMSK implementation. A generic floating-point GMSK reference would not match the phase quantization and filtering behavior closely enough.

`preamble_sample_indices.m`

Despite its filename, this function returns sample indices for positions where `bitMapMask == 1`, which are preamble bits. In the current `rx_chain.m`, these indices are used as LMS training samples:

```
trainSampleIdx = preamble_sample_indices(waveform.bitMapMask, SPS);
```

This means the current receiver trains the equalizer on preamble samples. The reference-generation helper `gmsk_modulate_burst_ref.m` can also produce indices for `bitMapMask == 3` training bits, but `rx_chain.m` currently uses the preamble-index helper.

`lms_equalizer_complex.m`

Production equalizer used by `rx_chain.m` and `sweep_equalizer_params.m`.

It is a complex baseband normalized LMS (NLMS) equalizer that operates before GMSK demapping. Inputs are:

- `rx`: normalized complex burst samples
- `s_ref`: ideal complex reference samples with the same length
- `trainSampleIdx`: sample locations where the reference is trusted
- `mu`: NLMS step size
- `eq_len`: FIR tap count
- `decisionDelay`: alignment delay between equalizer output and reference
- `verbose`: optional diagnostic printing flag

The equalizer computes:

```
u(n) = [rx(n), rx(n-1), ..., rx(n-eq_len+1)]
y(n) = w^H u(n)
e(n) = s_ref(n - decisionDelay) - y(n)
w <- w + (mu / (u^H u + eps)) u conj(e)
```

Only samples listed in `trainSampleIdx` update the taps. The equalizer still produces an output for all samples once the tap buffer is available.

Outputs are:

- `y`: equalized complex burst
- `w_final`: final complex tap vector
- `mse_curve`: per-update squared error

This is the equalizer that belongs to the real GMSK receiver.

`demod_aligned_burst.m`

Demaps an already aligned and equalized burst. Unlike `RxDemodulator`, it does not trim Gaussian group delay. It:

- Calls `GmskDemapper`.
- Samples every `samplesPerSymbol` samples starting from sample 1.
- Returns exactly `expectedSymbols` symbol metrics.

This avoids losing symbols after `rx_chain.m` has already extracted a burst using group-delay-aware timing.

`extract_payload.m`

Extracts encoded payload bits from a hard-decision burst bit vector by skipping the preamble and training chunks. This is useful for a hard-bit receiver path, but the current `rx_chain.m` mainly uses `waveform.bitMapMask == 2` directly on soft metrics.

`viterbi_soft_decoder.m`

Soft-decision Viterbi decoder matching the local rate-1/2 convolutional encoder. It:

- Uses `K = 7, 64` trellis states.
- Reshapes soft metrics into pairs.
- Computes branch metrics by correlating expected encoded bits with received soft values.
- Uses traceback length `TB_LEN`, currently 32 in `rx_chain.m`.
- Returns decoded information bits.

The metric convention assumes positive soft values favor bit 1 and negative soft values favor bit 0.

Equalization: `mathwork_lms_equalization.m` vs `lms_equalizer_complex.m`

These two files are related by concept only. They are not interchangeable.

`mathwork_lms_equalization.m`

This is a standalone educational LMS example. It appears to be adapted from a MathWorks-style demonstration and is not integrated into the GMSK receiver.

Characteristics:

- Script, not a function.
- Generates random binary data internally.
- Uses a raised-cosine transmit filter, not the project GMSK Gaussian filter.
- Uses a fixed real-valued FIR channel coefficient vector.
- Runs a basic LMS update over a synthetic reference/channel pair.
- Plots channel impulse response, equalizer response, equalized signal, and error behavior.
- Does not read `tx_data.csv`.
- Does not use `Waveform`, `GmskMapper`, `GmskDemapper`, `rx_chain`, or the GSM channel model.
- Does not decode packets or compute BER.

Its value in this repository is pedagogical: it shows the general idea of LMS equalization using a known reference signal. It is useful for understanding the algorithm, but it does not model this project's actual GMSK packet structure or receiver.

`lms_equalizer_complex.m`

This is the project-specific equalizer used in the real receive path.

Characteristics:

- Function, not a script.
- Operates on complex baseband GMSK burst samples.
- Uses the project-generated complex reference `s_ref`.
- Supports training only on selected known sample indices.
- Uses normalized LMS, which scales the update by input vector power.
- Supports `decisionDelay` to align equalizer output with the reference.
- Returns equalized samples for later GMSK demapping and Viterbi decoding.
- Is called directly by `rx_chain.m` and `sweep_equalizer_params.m`.

Practical Difference

Use `mathwork_lms_equalization.m` when you want to study a small generic LMS example in isolation.

Use `lms_equalizer_complex.m` when you want the actual receiver to work. It is aware of this project's complex samples, reference alignment, training indices, and burst-level processing.

The main algorithmic difference is also important:

- `mathwork_lms_equalization.m` uses a simple LMS update:

```
w <- w + mu * u * error
```

- `lms_equalizer_complex.m` uses a complex NLMS update:

```
w <- w + (mu / (u^H u + eps)) * u * conj(error)
```

The normalized update is usually more stable when received signal power varies, which is exactly what happens after fading and burst amplitude changes.

CSV and Comparison Utilities

`read_axi_stream_csv.m`

Reads CSV files with the header:

```
TDATA, TKEEP, TLAST
```

It unpacks:

- 32-bit `TDATA` words
- `TKEEP` byte-valid masks
- `TLAST` frame boundaries

It returns a struct containing raw words, keep masks, frame flags, unpacked bytes, MSB-first bits, and per-frame byte vectors.

`write_axi_stream_csv.m`

Writes a byte vector back into the same AXI4-Stream-like CSV format. It packs up to four bytes per row, writes an appropriate `TKEEP`, and sets `TLAST = 1` on the final row.

`compare_payload_bytes.m`

Compares transmitted and received payload bytes. It:

- Reads both CSV files.
- Pads or truncates both payloads to the expected payload size.
- Counts byte mismatches.
- Prints the first few mismatching byte positions and values.

Data and Generated Files

`tx_data.csv`

Input payload data in AXI4-Stream-like CSV format. `tx_chain.m` reads this file and pads/truncates its bytes to the waveform payload length.

`rx_data.csv`

Receiver output generated by `rx_chain.m`. It is overwritten whenever the receiver runs.

`Figure_1.png`

Saved plot image, likely from a BER or diagnostic run. It is not used by the simulation code.

Typical Usage

Run a single end-to-end link check:

```
cd('D:\Internship\GMSK\snr_vs_ber\real_curve')  
main_system
```

Run the BER sweep:

```
cd('D:\Internship\GMSK\snr_vs_ber\real_curve')  
plot_ber_vs_snr
```

Tune equalizer parameters:

```
cd('D:\Internship\GMSK\snr_vs_ber\real_curve')  
results = sweep_equalizer_params('tx_data.csv', 100);
```

Current Receiver Assumptions

- **SPS** = 4 throughout TX, channel timing, reference generation, and RX.
- **BT** = 0.35 and Gaussian span 8 symbols are shared between TX and RX reference generation.
- Only waveform ID 0 is implemented.
- Payload length is fixed to 34 information bytes.
- The receiver writes decoded output to **rx_data.csv**.
- **rx_chain.m** processes all detected bursts but BER code usually compares the first decoded burst.
- Equalization is performed on complex samples before demapping.
- Burst timing is based on soft preamble correlation from the demodulated full stream.

Development Notes

- Keep transmitter and receiver reference-generation parameters synchronized. A mismatch in **BT**, **SPS**, Gaussian span, LUT size, or mapper scaling changes the phase trajectory and can break LMS convergence.
- Be careful when changing burst structure in **Waveform.m**; update **build_packet.m**, payload extraction, reference generation, and receiver masks together.
- If equalizer performance changes, start with **sweep_equalizer_params.m** before changing receiver logic.
- If high-SNR BER does not improve, inspect preamble detection, burst alignment, equalizer MSE, and the sign convention of the soft metrics passed to **viterbi_soft_decoder.m**.